

小结

标准库容器是模板类型，用来保存给定类型的对象。在一个顺序容器中，元素是按顺序存放的，通过位置来访问。顺序容器有公共的标准接口：如果两个顺序容器都提供一个特定的操作，那么这个操作在两个容器中具有相同的接口和含义。

所有容器（除 array 外）都提供高效的动态内存管理。我们可以向容器中添加元素，而不必担心元素存储在哪里。容器负责管理自身的存储。vector 和 string 都提供更细致的内存管理控制，这是通过它们的 reserve 和 capacity 成员函数来实现的。

很大程度上，容器只定义了极少的操作。每个容器都定义了构造函数、添加和删除元素的操作、确定容器大小的操作以及返回指向特定元素的迭代器的操作。其他一些有用的操作，如排序或搜索，并不是由容器类型定义的，而是由标准库算法实现的，我们将在第 10 章介绍这些内容。

当我们使用添加和删除元素的容器操作时，必须注意这些操作可能使指向容器中元素的迭代器、指针或引用失效。很多会使迭代器失效的操作，如 insert 和 erase，都会返回一个新的迭代器，来帮助程序员维护容器中的位置。如果循环程序中使用了改变容器大小的操作，就要尤其小心其中迭代器、指针和引用的使用。

术语表

适配器 (adaptor) 标准库类型、函数或迭代器，它们接受一个类型、函数或迭代器，使其行为像另外一个类型、函数或迭代器一样。标准库提供了三种顺序容器适配器：stack、queue 和 priority_queue。每个适配器都在其底层顺序容器类型之上定义了一个新的接口。

数组 (array) 固定大小的顺序容器。为了定义一个 array，除了元素类型之外还必须给定大小。array 中的元素可以用其位置下标来访问。array 支持快速的随机访问。

begin 容器操作，返回一个指向容器首元素的迭代器，如果容器为空，则返回尾后迭代器。是否返回 const 迭代器依赖于容器的类型。

cbegin 容器操作，返回一个指向容器首元素的 const_iterator，如果容器为空，则返回尾后迭代器。

cend 容器操作，返回一个指向容器尾元素之后（不存在的）的 const_iterator。

容器 (container) 保存一组给定类型对象的类型。每个标准库容器类型都是一个模板类型。为了定义一个容器，我们必须指定保存在容器中的元素的类型。除了 array 之外，标准库容器都是大小可变的。

deque 顺序容器。deque 中的元素可以通过位置下标来访问。支持快速的随机访问。deque 各方面都与 vector 类似，唯一的差别是，deque 支持在容器头尾位置的快速插入和删除，而且在两端插入或删除元素都不会导致重新分配空间。

end 容器操作，返回一个指向容器尾元素之后（不存在的）元素的迭代器。是否返回 const 迭代器依赖于容器的类型。

forward_list 顺序容器，表示一个单向链表。forward_list 中的元素只能顺序访问。从一个给定元素开始，为了访问另一个元素，我们只能遍历两者之间的所有元素。forward_list 上的迭代器不支持递减运算 (--)。forward_list 支持任意位置的快速插入（或删除）操作。与其他容器不同，插入和删除发生在一个给定的

373 通过位置下标来访问。支持快速的随机访问。

迭代器之后的位置。因此，除了通常的尾后迭代器之外，`forward_list` 还有一个“首前”迭代器。在添加新元素后，原有的指向 `forward_list` 的迭代器仍有效。在删除元素后，只有原来指向被删元素的迭代器才会失效。

迭代器范围 (iterator range) 由一对迭代器指定的元素范围。第一个迭代器表示序列中第一个元素，第二个迭代器指向最后一个元素之后的位置。如果范围为空，则两个迭代器是相等的（反之亦然，如果两个迭代器不等，则它们表示一个非空范围）。如果范围非空，则必须保证，通过反复递增第一个迭代器，可以到达第二个迭代器。通过递增迭代器，序列中每个元素都能被访问到。

左闭合区间 (left-inclusive interval) 值范围，包含首元素，但不包含尾元素。通常表示为 $[i, j)$ ，表示序列从 i 开始（包含）直至 j 结束（不包含）。

list 顺序容器，表示一个双向链表。`list` 中的元素只能顺序访问。从一个给定元素开始，为了访问另一个元素，我们只能遍历两者之间的所有元素。`list` 上的迭代器既支持递增运算（++），也支持递减运算（--）。`list` 支持任意位置的快速插入（或删除）操作。当加入新元素后，迭代器仍然有效。当删除元素后，只有原来指向被删除元素的迭代器才会失效。

首前迭代器 (off-the-beginning iterator) 表示一个 `forward_list` 开始位置之前

（不存在的）元素的迭代器。是 `forward_list` 的成员函数 `before_begin` 的返回值。与 `end()` 迭代器类似，不能被解引用。

尾后迭代器 (off-the-end iterator) 表示范围中尾元素之后位置的迭代器。通常被称为“末尾迭代器”（end iterator）。

priority_queue 顺序容器适配器，生成一个队列，插入其中的元素不放在末尾，而是根据特定的优先级排列。默认情况下，优先级用元素类型上的小于运算符确定。

queue 顺序容器适配器，生成一个类型，使我们能将新元素添加到末尾，从头部删除元素。

顺序容器 (sequential container) 保存相同类型对象有序集合的类型。顺序容器中的元素通过位置来访问。

stack 顺序容器适配器，生成一个类型，使我们只能在其一端添加和删除元素。

vector 顺序容器。`vector` 中的元素可以通过位置下标访问。支持快速的随机访问。我们只能在 `vector` 末尾实现高效的元素添加/删除。向 `vector` 添加元素可能导致内存空间的重新分配，从而使所有指向 `vector` 的迭代器失效。在 `vector` 内部添加（或删除）元素会使所有指向插入（或删除）点之后元素的迭代器失效。

第 10 章

泛型算法

内容

10.1 概述.....	336
10.2 初识泛型算法	338
10.3 定制操作.....	344
10.4 再探迭代器	357
10.5 泛型算法结构	365
10.6 特定容器算法	369
小结	371
术语表.....	371

标准库容器定义的操作集合惊人得小。标准库并未给每个容器添加大量功能，而是提供了一组算法，这些算法中的大多数都独立于任何特定的容器。这些算法是通用的（generic，或称泛型的）：它们可用于不同类型的容器和不同类型的元素。

泛型算法和关于迭代器的更多细节，构成了本章的主要内容。